

**Course: Fundamentals of Generative AI and Large Language
Models: Theory and Practice**

**Week 3: Variational Autoencoders (VAEs):
Foundations**

Today's Session Overview

➤ Reparameterization Trick

NPTREL

Variational Autoencoders (VAEs): Loss Function

$$L(x) = \underbrace{-\mathbb{E}_{q_{\phi}(z|x)}[\log p_{\theta}(x|z)]}_{\text{Reconstruction}} + \underbrace{KL(q_{\phi}(z|x) \parallel p(z))}_{\text{Regularization}}$$

Reconstruction



MSE for continuous-valued data
Binary Cross-Entropy (BCE) for binary data

The latent vector z is sampled from:

$$z \sim q_{\phi}(z|x) = \mathcal{N}(\mu(x), \sigma^2(x))$$

Regularization



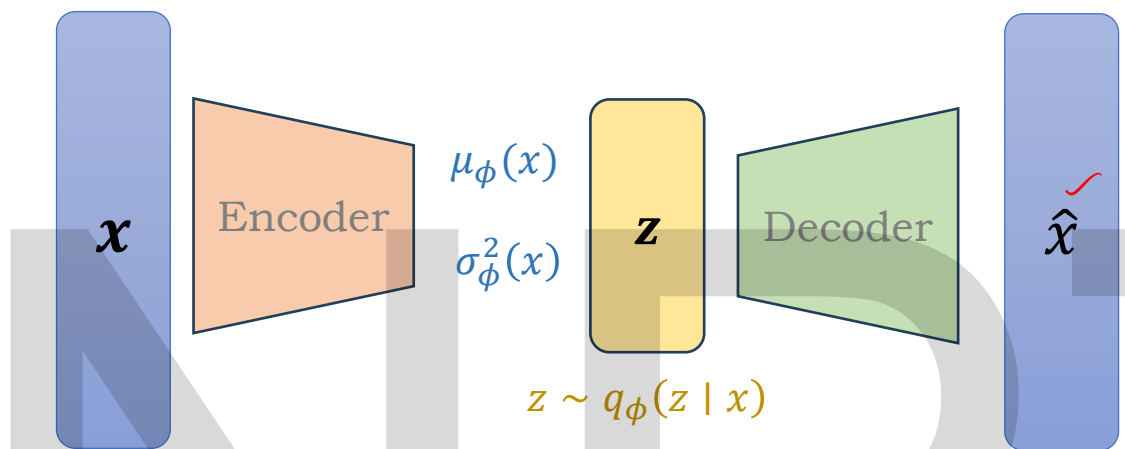
$$L = L_{KL}(\mathcal{N}(\mu, \sigma^2) \parallel \mathcal{N}(0,1))$$
$$L_{KL} = \frac{1}{2} [\mu^2 + \sigma^2 - \log \sigma^2 - 1]$$

When both distributions are Gaussian, the KL divergence has a closed-form expression

Since sampling is a random operation, Randomness breaks gradient flow, making backpropagation impossible

So, the question is: **How do we compute gradients when part of the network involves sampling?**

This motivates the Reparameterization Trick



$p_{data}(x)$

For an input x , the encoder outputs the parameters of an approximate posterior distribution:

$$q_{\phi}(z | x) = \mathcal{N}(\mu_{\phi}(x), \sigma_{\phi}^2(x))$$

The latent vector z is sampled from this distribution:

$$z \sim q_{\phi}(z | x) \checkmark$$

or

$$z \sim \mathcal{N}(\mu_{\phi}(x), \sigma_{\phi}^2(x))$$

The sampled latent vector z is then passed to the decoder.

The decoder produces the reconstruction:

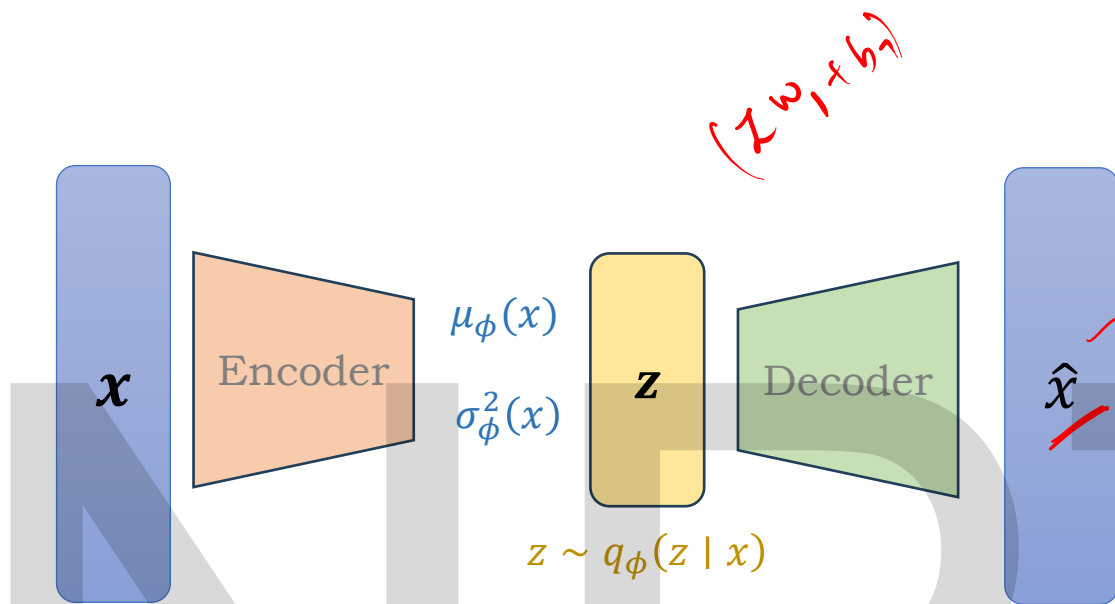
$$\hat{x} = f_{\theta}(z)$$

Here, z is a sampled latent vector

f_{θ} is a decoder neural network

θ represents all decoder parameters, such as weights and biases. The decoder parameters θ are learned so that $\hat{x}$ becomes as close as possible to x .

$\hat{x}$ is the reconstructed output produced by the decoder



$p_{data}(x)$

In an MLP decoder, the sampled latent vector z is passed as the input to a normal feed-forward neural network.

Step 1: First decoder layer:

$$h_1 = \text{ReLU}(W_1 z + b_1) \quad \checkmark$$

The decoder takes the latent vector z , multiplies it with weight matrix W_1 , adds bias b_1 , and applies ReLU activation. So h_1 is the first hidden representation inside the decoder.

Step 2: Output reconstruction layer:

$$\hat{x} = f(W_2 h_1 + b_2)$$

The hidden representation h_1 is passed to the output layer to reconstruct the input.

The activation function $f(\cdot)$ depends on the type of data:

Sigmoid (σ): for binary data or pixel values normalized to $[0,1]$.

Linear activation: for continuous-valued data.

Decoder parameters:

$$\theta = \{W_1, b_1, W_2, b_2\} \quad \checkmark$$

Once $\hat{x}$ is obtained, compute loss:

Loss ($\hat{x}, x$) Then update the parameters.

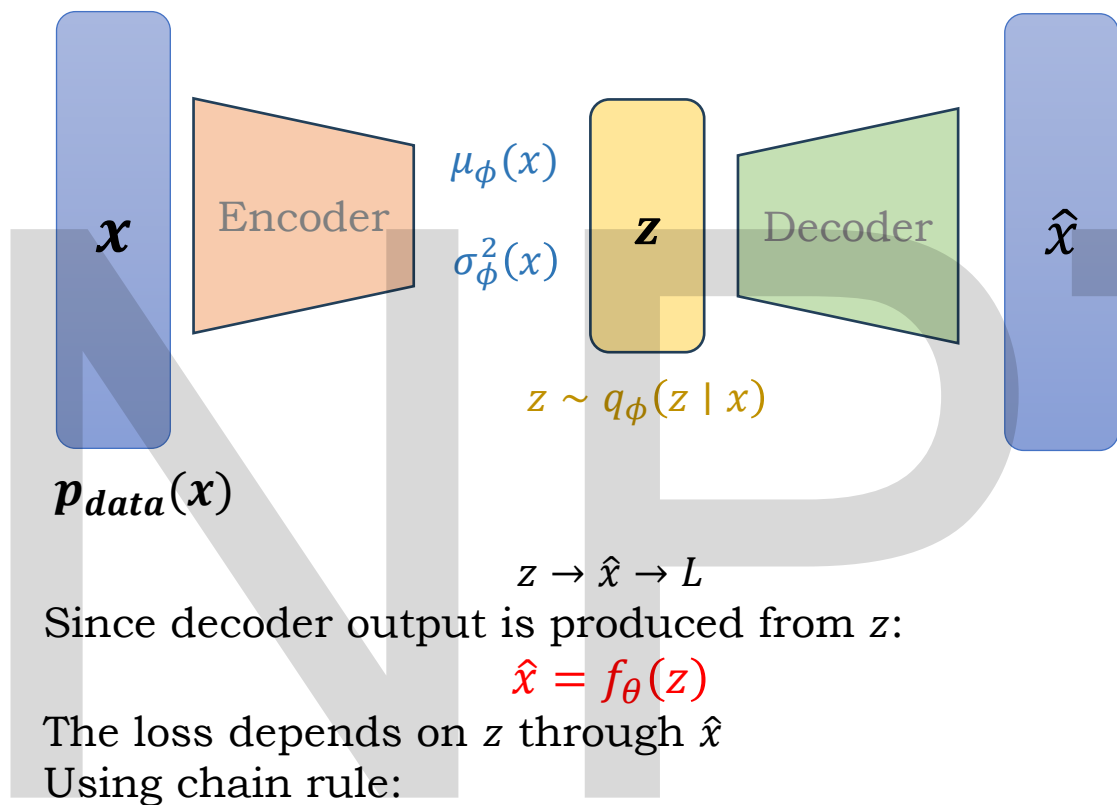
For the output reconstruction layer:

$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial \hat{x}} \cdot \frac{\partial \hat{x}}{\partial W_2} \quad \frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial \hat{x}} \cdot \frac{\partial \hat{x}}{\partial b_2} \quad \checkmark$$

For the first decoder layer:

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial \hat{x}} \cdot \frac{\partial \hat{x}}{\partial h_1} \cdot \frac{\partial h_1}{\partial W_1} \quad \frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial \hat{x}} \cdot \frac{\partial \hat{x}}{\partial h_1} \cdot \frac{\partial h_1}{\partial b_1}$$

Therefore, gradients with respect to decoder parameters are computed using standard backpropagation. Hence, the decoder side is not the main difficulty in VAEs.



$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial \hat{x}} \cdot \frac{\partial \hat{x}}{\partial z}$$

z is treated as a numeric input to the decoder.
The decoder is differentiable with respect to its input.
 $\therefore$, gradients can flow backward from the reconstruction loss up to z .

However, the difficulty is not in the decoder.

The real problem is how to pass gradients from z back to the encoder outputs $\mu_\phi(x)$ and $\sigma_\phi(x)$.

But the problem starts when we try to update the encoder.

The encoder produces:

$$q_\phi(z | x) = \mathcal{N}(\mu_\phi(x), \sigma_\phi^2(x))$$

Then the latent vector is sampled as:

$$z \sim q_\phi(z | x)$$

To update the encoder, we need gradients with respect to $\mu_\phi(x)$ and $\sigma_\phi(x)$:

$$\frac{\partial L}{\partial \mu_\phi} \text{ and } \frac{\partial L}{\partial \sigma_\phi}$$

Using chain rule:

$$\frac{\partial L}{\partial \mu_\phi} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial \mu_\phi} \quad \frac{\partial L}{\partial \sigma_\phi} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial \sigma_\phi}$$

Because z is obtained through random sampling:

$$z \sim \mathcal{N}(\mu_\phi, \sigma_\phi^2) \checkmark$$

So z is not directly expressed as a differentiable function of μ_ϕ and σ_ϕ .

Therefore, direct backpropagation through sampling is not possible.

How can we sample z so that gradients can flow back to μ and σ ?

Instead of directly sampling:

$$z \sim \mathcal{N}(\mu_\phi(x), \sigma_\phi^2(x))$$

we rewrite the sampling operation as:

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

Here:

$$\mu_\phi(x) \text{ and } \sigma_\phi(x)$$

are produced by the encoder and depend on x and encoder parameters ϕ .

ϵ is not learned by the network but is sampled from a standard normal distribution whose values are independent of μ and σ . ϵ does not depend on ϕ . The network learns μ and σ , not ϵ .

Therefore, during backpropagation, ϵ is treated as a constant.

This makes z a differentiable function of $\mu_\phi(x)$ and $\sigma_\phi(x)$.

Reparameterization Trick

The reparameterization trick separates randomness from the encoder parameters:

$$z = \mu + \sigma \odot \epsilon$$

Now gradients can flow from the decoder output back to the encoder.

Compute the partial derivative w.r.t. μ

$$z = \mu + \sigma \odot \epsilon$$

Differentiate w.r.t. μ :

$$\frac{\partial z}{\partial \mu} = \frac{\partial}{\partial \mu}(\mu) + \frac{\partial}{\partial \mu}(\sigma \epsilon)$$

$$\Rightarrow \frac{\partial \mu}{\partial \mu} = 1$$

Since $\sigma \odot \epsilon$ does **not** contain $\mu \rightarrow$ derivative is 0

$$\frac{\partial z}{\partial \mu} = 1 + 0$$

$$\boxed{\frac{\partial z}{\partial \mu} = 1}$$

Compute the partial derivative w.r.t. σ

$$z = \mu + \sigma \odot \epsilon$$

Differentiate w.r.t. σ :

$$\frac{\partial z}{\partial \sigma} = \frac{\partial}{\partial \sigma}(\mu) + \frac{\partial}{\partial \sigma}(\sigma \epsilon)$$

Since μ does not depend on σ and ϵ is treated as constant

$$\begin{aligned} \frac{\partial}{\partial \sigma}(\sigma \epsilon) &= \epsilon \\ \frac{\partial z}{\partial \sigma} &= 0 + \epsilon \end{aligned}$$

$$\boxed{\frac{\partial z}{\partial \sigma} = \epsilon}$$

Quick Summary:

The decoder gives:

$$\hat{x} = f_{\theta}(z)$$

Therefore, the loss depends on z through $\hat{x}$:

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial \hat{x}} \cdot \frac{\partial \hat{x}}{\partial z}$$

Using reparameterization:

$$z = \mu_{\phi}(x) + \sigma_{\phi}(x) \odot \epsilon$$

So,

$$\frac{\partial z}{\partial \mu_{\phi}} = 1$$

$$\frac{\partial z}{\partial \sigma_{\phi}} = \epsilon$$

Once the gradient reaches the latent vector z , it must flow back to the encoder outputs: the mean $\mu_{\phi}(x)$ and the standard deviation $\sigma_{\phi}(x)$.

So:

$$L \rightarrow \hat{x} \rightarrow z \rightarrow \mu_{\phi}(x), \sigma_{\phi}(x)$$

Thus, the reparameterization trick enables gradients to flow from the loss all the way back to the encoder parameters, allowing end-to-end training of the VAE.

For μ_{ϕ} :

$$L \rightarrow \hat{x} \rightarrow z \rightarrow \mu_{\phi}$$

$$\frac{\partial L}{\partial \mu_{\phi}} = \frac{\partial L}{\partial \hat{x}} \cdot \frac{\partial \hat{x}}{\partial z} \cdot \frac{\partial z}{\partial \mu_{\phi}}$$

Since

$$\frac{\partial z}{\partial \mu_{\phi}} = 1$$

we get

$$\frac{\partial L}{\partial \mu_{\phi}} = \frac{\partial L}{\partial \hat{x}} \cdot \frac{\partial \hat{x}}{\partial z}$$

Finally, the gradients reach the encoder outputs $\mu_{\phi}(x)$ and $\sigma_{\phi}(x)$, and can propagate to the encoder parameters ϕ using standard backpropagation.

$$L \rightarrow \hat{x} \rightarrow z \rightarrow \mu_{\phi}(x), \sigma_{\phi}(x) \rightarrow \phi$$

Mathematically:

$$\frac{\partial L}{\partial \phi} = \frac{\partial L}{\partial \mu_{\phi}} \frac{\partial \mu_{\phi}}{\partial \phi} + \frac{\partial L}{\partial \sigma_{\phi}} \frac{\partial \sigma_{\phi}}{\partial \phi}$$

For σ_{ϕ} :

$$L \rightarrow \hat{x} \rightarrow z \rightarrow \sigma_{\phi}$$

$$\frac{\partial L}{\partial \sigma_{\phi}} = \frac{\partial L}{\partial \hat{x}} \cdot \frac{\partial \hat{x}}{\partial z} \cdot \frac{\partial z}{\partial \sigma_{\phi}}$$

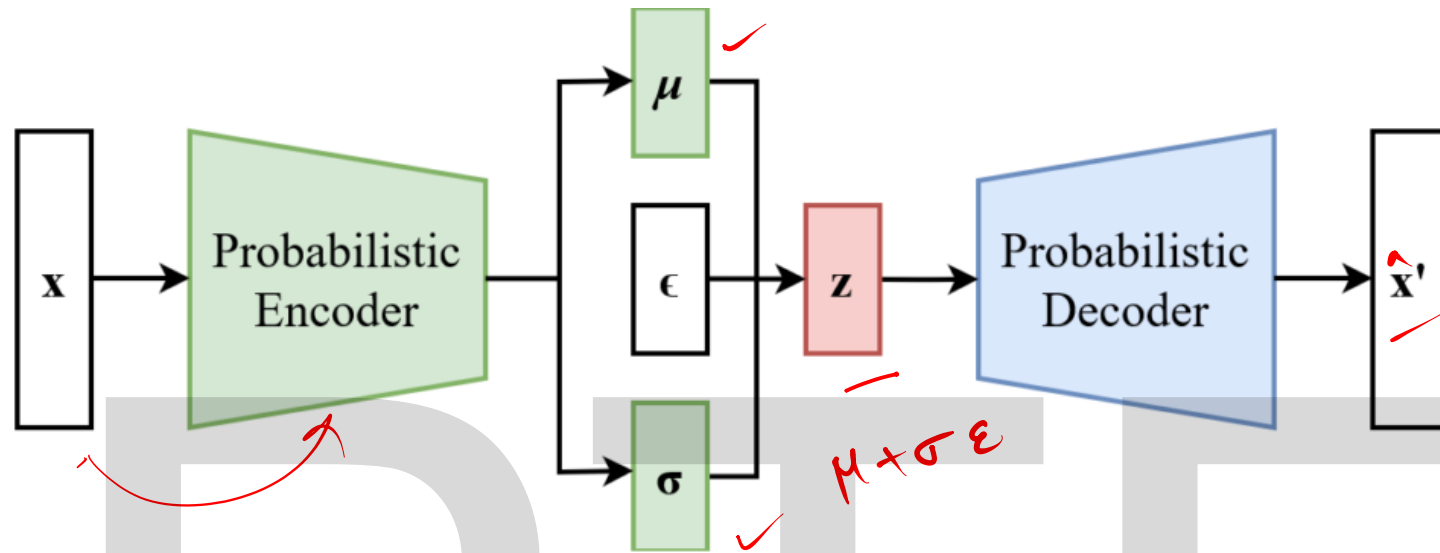
Since

$$\frac{\partial z}{\partial \sigma_{\phi}} = \epsilon$$

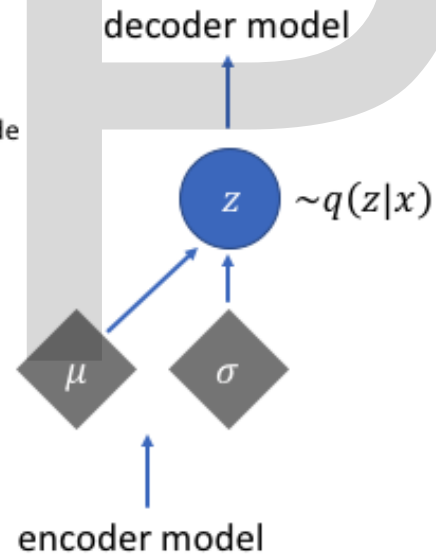
we get

$$\frac{\partial L}{\partial \sigma_{\phi}} = \frac{\partial L}{\partial \hat{x}} \cdot \frac{\partial \hat{x}}{\partial z} \cdot \epsilon$$

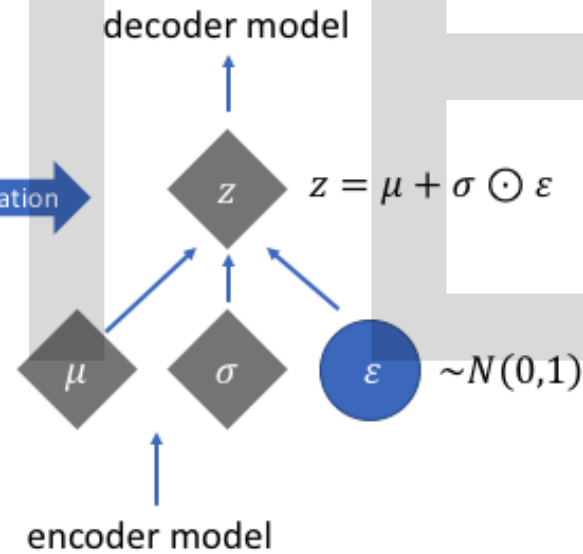
VAE Architecture



Reparameterization



reparameterization



Before reparameterization:

$$z \sim q_{\phi}(z | x) \Rightarrow z = \text{random function of } \phi$$

After reparameterization:

$$\epsilon \sim \mathcal{N}(0, I) \text{ (random but independent)}$$

$$z = \mu_{\phi}(x) + \sigma_{\phi}(x) \odot \epsilon \text{ (deterministic)}$$

Now z is differentiable w.r.t. ϕ

Next Session

- Understand VAE working using a numerical example (full forward pass)

NIPTEEL

NIPTELL

Thank you